

# Ingeniería del software de componentes y sistemas distribuidos

PR3 - Calidad del software distribuido: inherente, transversal y crítico

## Índice

|                                                                |           |
|----------------------------------------------------------------|-----------|
| <b>Ejercicio 1: Estrategia de pruebas (20 %)</b> .....         | <b>2</b>  |
| Apartado 1 (15 %).....                                         | 2         |
| Apartado 2 (5 %).....                                          | 4         |
| <b>Ejercicio 2: Pruebas unitarias y reporting (20 %)</b> ..... | <b>5</b>  |
| Apartado 1 (10 %).....                                         | 5         |
| Apartado 2 (10 %).....                                         | 5         |
| <b>Ejercicio 3: Pruebas de contrato (35 %)</b> .....           | <b>10</b> |
| <b>Ejercicio 4: Pruebas de mutación (25 %)</b> .....           | <b>19</b> |

# Ejercicio 1: Estrategia de pruebas (20 %)

El archivo `CourseApplicationTests.java` solo contiene una prueba que verifica que el contexto de Spring arranque. Teniendo en cuenta la arquitectura de microservicios de la aplicación:

## Apartado 1 (15 %)

Define una **estrategia de pruebas para esta aplicación**, indicando qué niveles de prueba existen, qué tipos de pruebas son necesarios para cada nivel y qué aspectos concretos cubrirían.

Justifica en detalle las decisiones y aporta adicionalmente un diagrama resumen en la que se indique para cada nivel de pruebas qué tipos de pruebas se incluyen y qué librerías o frameworks de pruebas automáticas se proponen.

---

Estrategia de pruebas propuesta:

### 1. Nivel estático y arquitectural, como capa transversal.

Aquí entran las reglas de arquitectura con ArchUnit. Su objetivo es comprobar que la estructura de capas se respeta. El controlador depende del servicio, el servicio del dominio y repositorios, y el dominio no conoce infraestructura. También sirve para evitar acoplamientos indebidos, por ejemplo que la capa de dominio acabe usando RestTemplate o clases de persistencia. Este nivel no valida comportamiento funcional, pero sí previene regresiones de diseño que en microservicios suelen degradar mucho la mantenibilidad.

### 2. Nivel unitario.

Es el nivel más barato y el que más valor aporta al principio. Aquí usaría JUnit 5, Mockito y AssertJ. El foco debe estar en la lógica pura del dominio y del servicio, aislando dependencias con mocks.

### 3. Nivel de integración.

Este nivel comprueba que las piezas que hablan con infraestructura funcionan juntas de verdad. Aquí propondría Spring Boot Test, DataJpaTest, MockMvc o RestAssured, y Testcontainers para la base de datos. Si hay dependencias HTTP externas, usaría WireMock o MockWebServer; si hay mensajería, Testcontainers para Kafka y spring-kafka-test. En microservicios este nivel es imprescindible porque los fallos más caros suelen aparecer en la frontera entre dominio e infraestructura: mapeo JPA, consultas, serialización JSON, configuración de URLs, manejo de estados transaccionales y código REST.

### 4. Nivel de contrato.

En una arquitectura de microservicios este nivel es crítico porque el sistema deja de ser monolítico y las interfaces entre servicios pasan a ser puntos de fallo. Aquí propondría Pact o Spring Cloud Contract para contratos HTTP, y si hubiera eventos, contratos de mensajes para

Kafka. En el microservicio de Cursos esto afecta sobre todo a las llamadas al servicio de usuarios y al servicio de microcredenciales, que en el código se usan desde la capa de servicio con RestTemplate.

## 5. Nivel E2E (extremo a extremo).

Este nivel debe ser reducido, porque es el más caro y frágil, pero sirve para validar escenarios de negocio completos con varios servicios desplegados. Aquí usaría REST Assured, Postman/Newman o pruebas automatizadas sobre un entorno levantado con contenedores



Diagrama a modo de resumen

## Apartado 2 (5 %)

Resume qué pruebas aplicarías al microservicio de Cursos en cada nivel y tipo definido en la estrategia.

---

Desglose de las pruebas al microservicio de Cursos en cada nivel y tipo:

- Unitarias: Probaría CourseServiceImpl y EnrollmentServiceImpl con mocks de CourseRepository, EnrollmentRepository, UserRepository y los clientes externos. Los casos clave a destacar en estos archivos son la creación de curso con instructor válido o inválido, apertura y cierre de matrícula según estado, matrícula repetida, cierre de curso y cambio de estado de matrículas, y errores cuando no existe curso o estudiante.
- Integración: Probaría CourseRepositoryImpl y EnrollmentRepositoryImpl contra una base real de pruebas, idealmente con Testcontainers y PostgreSQL; además, probaría CourseRestController con MockMvc o RestAssured para verificar rutas, validación de entrada, conversiones JSON y códigos HTTP. También validaría la integración con las llamadas HTTP que usa CourseServiceImpl para usuarios y microcredenciales, simulando el otro lado con WireMock o con tests de contrato.
- Contrato: Definiría contratos para el consumo del servicio de usuarios y del servicio de microcredenciales, ya que Cursos depende de ambos. Aquí se comprobaría que las respuestas que Cursos espera siguen existiendo y tienen el formato correcto. Si el microservicio de usuarios cambia la estructura de GetUserResponse o el de microcredenciales cambia el endpoint, este nivel lo detecta antes de desplegar.
- E2E: Automatizaría solo los recorridos más representativos del ciclo de vida del curso, como el alta de curso, la apertura de inscripciones, el alta de matrícula, el cierre de inscripciones, el cierre de actas y el cierre final del curso. Es suficiente con pocos escenarios bien elegidos porque el objetivo aquí es detectar fallos de integración global, no agotar combinaciones.

# Ejercicio 2: Pruebas unitarias y reporting (20 %)

## Apartado 1 (10 %)

Escribe una prueba unitaria para `CourseServiceImpl.createCourse()` usando JUnit5, Mockito y AssertJ. La prueba debe verificar los siguientes escenarios:

- a) El instructor existe → el curso se crea con estado DRAFT
- b) El instructor no existe → se lanza `UserNotFoundException` y no se invoca el método `createCourse` del `CourseRepository`

Justifica qué dependencias necesitas mockear y por qué no puedes instanciar `CourseServiceImpl` directamente sin mocks.

### Notas:

En las pruebas unitarias no hacemos uso de ninguna infraestructura ni dependencia real. No usamos comunicación HTTP, servidores, bases de datos, servicios externos u otros microservicios. Todo aspecto relacionado con dependencias debe simularse (con Mockito), excepto las clases de dominio (`Course`, `Enrollment`, ...).

---

El test creado para este apartado se llama `CourseServiceImplTest`. He mockeado `CourseRepository`, `UserRepository`, `EnrollmentRepository` y `EnrollmentService` porque `CourseServiceImpl` los recibe por inyección de constructor y una prueba unitaria no debe tocar infraestructura real ni dependencias externas. Aunque `createCourse()` solo usa `UserRepository` y `CourseRepository`, no se puede instanciar `CourseServiceImpl` directamente sin proporcionar también las otras dependencias obligatorias; además, esas colaboraciones deben simularse para aislar el comportamiento del servicio.

`CourseServiceImpl` no se mockea porque es nuestro sujeto de prueba (System Under Test), y las clases de dominio, como `Course`, se instancian de forma real porque representan estado y no contienen dependencias de infraestructura.

## Apartado 2 (10 %)

Utilizando como base las pruebas realizadas en el apartado anterior, debes configurar el adaptador de Allure correspondiente a Junit5 (<https://allurereport.org/docs/junit5/>) para transformar los logs de consola en un informe visual en HTML.

En el informe de entrega PDF se debe incluir:

- Pasos realizados para integrar el proyecto con Allure, junto con fragmentos relevantes de los archivos de configuración y del código de pruebas si se han modificado.
- Descripción breve del proceso para generar el archivo HTML de resultados y de su contenido (Overview, Timeline), apoyada por capturas de pantalla como evidencia.

Entrega adicionalmente un ZIP que incluya los archivos HTML y ficheros relacionados generados con Allure.

---

Para integrar Allure en el proyecto, tenemos que integrar el adaptador `allure-junit5` en el gestor de dependencias del proyecto (en este caso, Maven). Para ello, añadimos el siguiente código al archivo `pom.xml` del microservicio de Courses:

```
<properties>
  <java.version>11</java.version>
  <allure.version>2.35.2</allure.version>
</properties>
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>io.qameta.allure</groupId>
      <artifactId>allure-bom</artifactId>
      <version>${allure.version}</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
...
  <dependency>
    <groupId>io.qameta.allure</groupId>
    <artifactId>allure-junit5</artifactId>
    <scope>test</scope>
  </dependency>
...
  <dependency>
    <groupId>io.qameta.allure</groupId>
    <artifactId>allure-junit5</artifactId>
```

```
<scope>test</scope>
</dependency>
```

Para enriquecer el reporte visual, podemos añadir anotaciones de Allure en nuestra clase `CourseServiceImplTest`.

```
import io.qameta.allure.Epic;
import io.qameta.allure.Feature;
import io.qameta.allure.Story;

@Epic("Gestión de Cursos")
@Feature("Creación de Cursos")
@ExtendWith(MockitoExtension.class)
class CourseServiceImplTest {

    @Test
    @Story("Creación exitosa de curso en estado DRAFT")
    @DisplayName("El instructor existe -> el curso se crea con estado DRAFT")
    void
createCourse_WhenInstructorExists_ShouldCreateCourseWithDraftStatus() { ... }

    // ...
}
```

Podemos incluso enriquecer el reporte HTML de forma dinámica durante la ejecución de la prueba:

```
private void attachCourseContext(String title, Course course) {
    Allure.step(title);
    Allure.attachment(
        "Contexto del curso",
        String.format(
            "Instructor: %s\nTítulo: %s\nEstado inicial: %s\nModo: %s\nIdioma: %s\nUbicación: %s\n",
            course.getInstructor(),
            course.getTitle(),
            course.getStatus(),
            course.getMode(),
            course.getLanguage(),
            course.getLocation()
        )
    );
}
```

```
}
```

Y llamar a dicha función en los tests:

```
@Test
@DisplayName("Fallar cuando el instructor no existe")
@Story("Intento de creación con instructor inexistente")
void
createCourse_whenInstructorDoesNotExist_shouldThrowAndNotCreateC
ourse() {
    Course course =
buildCourse("missing-instructor@example.com");
    attachCourseContext("Escenario de fallo por instructor
inexistente", course);
    // ...
}
```

Para generar el archivo HTML de resultados y de su contenido, primero ejecutamos las pruebas para generar los resultados en crudos, los cuales se almacenan en archivos JSON en la carpeta `target/allure-results`. Esto se logra ejecutando: `mvn clean test`.

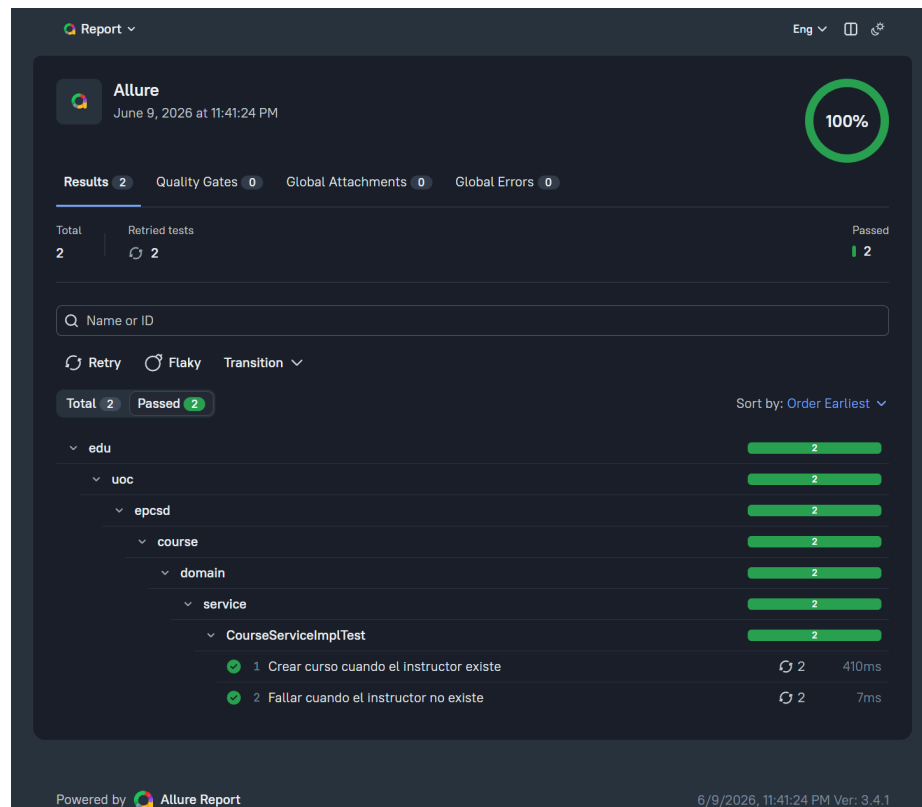
A continuación, compilamos esos resultados en un reporte HTML estático utilizando el comando: `mvn allure:report`. El reporte final se genera dentro del directorio `target/site/allure-maven-plugin/`.

El informe HTML incluye dos pestañas principales:

- **Pestaña Report:**

Esta pantalla presenta un panel de control a modo de resumen general con métricas clave y gráficos circulares interactivos. Permite ver de un vistazo el porcentaje total de pruebas exitosas (verdes), fallidas (rojas), rotas (naranjas) o ignoradas (grises). También muestra tendencias de ejecuciones pasadas y una categorización rápida de defectos si los hubiera.

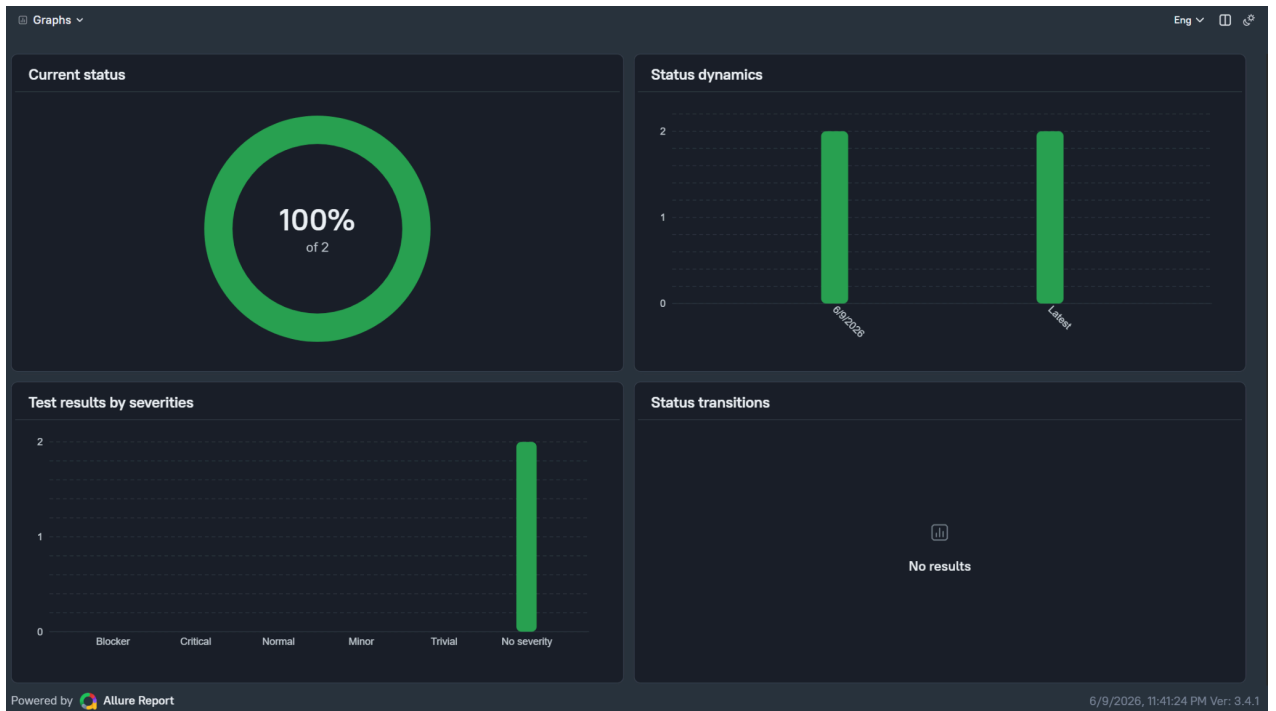




Pestaña Report

- **Pestaña Graphs:**

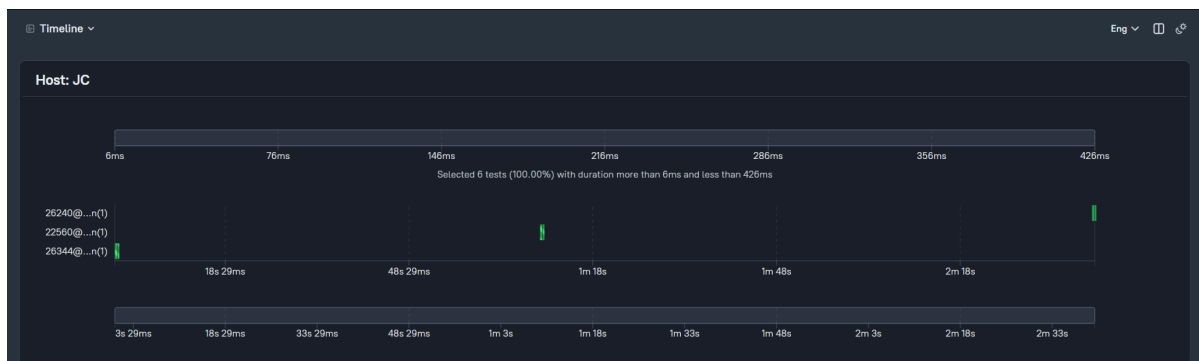
Esta sección es puramente analítica, diseñada para ofrecer diferentes perspectivas visuales sobre la salud y el rendimiento de la suite de pruebas. A diferencia de la pestaña Overview, la cual da un resumen general y rápido, Graphs desglosa los datos en métricas específicas que ayudan a los equipos de desarrollo a identificar patrones, cuellos de botella y niveles de riesgo.



Pestaña Graphs

- **Pestaña Timeline:**

Esta vista ilustra la ejecución temporal de los tests distribuidos a lo largo del tiempo de procesamiento y ordenados por los hilos de la JVM. Es particularmente útil para identificar cuellos de botella en el rendimiento de los tests o para entender qué pruebas se ejecutaron en paralelo y cuánto tardó exactamente cada una, como nuestros mocks de `CourseServiceImplTest`.



Pestaña Timeline

## Ejercicio 3: Pruebas de contrato (35 %)

Agregar pruebas de contratos es clave para garantizar que los servicios y microservicios funcionen correctamente juntos.

El objetivo de este ejercicio es generar pruebas automáticas de contrato utilizando Pact para validar los contratos establecidos entre CourseService (como consumidor) y MicroCredentialsService (como productor).

- a) Escenario 1. Creación exitosa de microcredenciales (/microcredentials/1/create): 201 Created o 200 OK
- b) Escenario 2. Curso no encontrado al crear unas microcredenciales (/microcredentials/999/create): 400

Debéis realizar:

- La implementación de la prueba de Pact en el lado del consumidor donde se definen las interacciones para los dos escenarios solicitados (debe incluirse la configuración del PactBuilder y el uso del servidor mock).
- Generar con Pact el archivo JSON que contiene las expectativas del contrato
- La implementación de la prueba de Pact en el lado del proveedor.
- Informe de ejecución con capturas o logs:
  - **Lado Consumidor:** Evidencia de que las pruebas pasaron y el archivo JSON se generó correctamente.
  - **Lado Proveedor:** Evidencia de que el proveedor validó con éxito el contrato enviado por el consumidor.

### Notas:

Podéis usar como referencia para ayudaros a desarrollar las pruebas:

- <https://www.baeldung.com/pact-junit-consumer-driven-contracts>

---

### CourseServicePactConsumerTest:

```
package edu.uoc.epcsd.course.domain.service;

import au.com.dius.pact.consumer.MockServer;
import au.com.dius.pact.consumer.dsl.PactBuilder;
import au.com.dius.pact.consumer.junit5.PactConsumerTestExt;
import au.com.dius.pact.consumer.junit5.PactTestFor;
import au.com.dius.pact.core.model.annotations.Pact;
import au.com.dius.pact.core.model.V4Pact;
import edu.uoc.epcsd.course.domain.Course;
import edu.uoc.epcsd.course.domain.CourseStatus;
import edu.uoc.epcsd.course.domain.repository.CourseRepository;
```

```

import
edu.uoc.epcsd.course.domain.repository.EnrollmentRepository;
import edu.uoc.epcsd.course.domain.repository.UserRepository;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.springframework.test.util.ReflectionTestUtils;

import java.util.ArrayList;
import java.util.Optional;

import static org.assertj.core.api.Assertions.assertThat;
import static
org.assertj.core.api.Assertions.assertThatThrownBy;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.mock;
import static org.mockito.Mockito.verify;
import static org.mockito.Mockito.when;

@ExtendWith(PactConsumerTestExt.class)
class CourseServicePactConsumerTest {

    private final CourseRepository courseRepository =
mock(CourseRepository.class);
    private final UserRepository userRepository =
mock(UserRepository.class);
    private final EnrollmentRepository enrollmentRepository =
mock(EnrollmentRepository.class);
    private final EnrollmentService enrollmentService =
mock(EnrollmentService.class);

    @Pact(consumer = "CourseService", provider =
"MicroCredentialsService")
    V4Pact microcredentialCreationContract(PactBuilder
builder) {
        builder.expectsToReceiveHttpInteraction("successful
microcredential creation for course 1", interaction ->
interaction
            .state("course 1 exists and is closed")
            .withRequest(request -> request
                .method("POST")
                .path("/microcredentials/1/create"))
            .willRespondWith(response -> response
                .status(201)
                .body("true"))));
    }

```

```

        builder.expectsToReceiveHttpInteraction("failed
microcredential creation for missing course 999", interaction ->
interaction
        .state("course 999 does not exist")
        .withRequest(request -> request
            .method("POST")
            .path("/microcredentials/999/create"))
        .willRespondWith(response -> response
            .status(400)));
    return builder.toPact();
}

@Test
@PactTestFor(pactMethod = "microcredentialCreationContract")
void
closeCourse_shouldMatchSuccessAndBadRequestContracts(MockServer
mockServer) {
    CourseServiceImpl courseService =
createCourseService(mockServer.getUrl());

    Course closedCourse = buildCourse(1L);

    when(courseRepository.getCourseById(1L)).thenReturn(Optional.of(
closedCourse));

    when(courseRepository.closeEnrollmentCourse(any(Course.class))).
thenReturn(1L);

    Long result = courseService.closeCourse(1L);

    assertThat(result).isEqualTo(1L);
    verify(enrollmentService).closeCourse(1L);

    Course missingCourse = buildCourse(999L);

    when(courseRepository.getCourseById(999L)).thenReturn(Optional.o
f(missingCourse));

    when(courseRepository.closeEnrollmentCourse(any(Course.class))).
thenReturn(999L);

    assertThatThrownBy(() ->
courseService.closeCourse(999L))

```

```

.isInstanceOf(org.springframework.web.client.HttpClientErrorException.BadRequest.class);
    }

    private CourseServiceImpl createCourseService(String
baseUrl) {
        CourseServiceImpl courseService = new CourseServiceImpl(
            courseRepository,
            userRepository,
            enrollmentRepository,
            enrollmentService);
        ReflectionTestUtils.setField(
            courseService,
            "microcredentialServiceUrl",
            baseUrl +
"/microcredentials/{courseId}/create");
        return courseService;
    }

    private Course buildCourse(Long courseId) {
        Course course = Course.builder()
            .id(courseId)
            .instructor("instructor@example.com")
            .enrollment(new ArrayList<>())
            .title("Course title")
            .description("Course description")
            .enrollmentStartDate(new java.util.Date())
            .enrollmentEndDate(new java.util.Date())
            .mode("ONLINE")
            .price(100L)
            .objectives("Objectives")
            .methology("Methodology")
            .duration(10L)
            .language("EN")
            .location("Remote")
            .build();
        course.setStatus(CourseStatus.PENDING_CLOSUE);
        return course;
    }
}

```

```

[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.416 s - in edu.uoc.epcsd.course.domain.service.CourseServicePactConsumerTest
[INFO] Results:
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 4.361 s
[INFO] Finished at: 2026-06-10T00:30:04+02:00
[INFO]

```

Tests pasando correctamente

Al ejecutar el test, se genera el siguiente JSON:

[CourseService-MicroCredentialsService.json](#)

```

{
  "consumer": {
    "name": "CourseService"
  },
  "interactions": [
    {
      "description": "successful microcredential creation for
course 1",
      "key": "dad5ee1a",
      "pending": false,
      "providerStates": [
        {
          "name": "course 1 exists and is closed"
        }
      ],
      "request": {
        "method": "POST",
        "path": "/microcredentials/1/create"
      },
      "response": {
        "body": {
          "content": true,
          "contentType": "application/json",
          "encoded": false
        },
        "headers": {
          "content-type": [
            "application/json"
          ]
        },
        "status": 201
      }
    ]
  }
}

```

```

        "transport": "http",
        "type": "Synchronous/HTTP"
    },
    {
        "description": "failed microcredential creation for
missing course 999",
        "key": "5806478b",
        "pending": false,
        "providerStates": [
            {
                "name": "course 999 does not exist"
            }
        ],
        "request": {
            "method": "POST",
            "path": "/microcredentials/999/create"
        },
        "response": {
            "status": 400
        },
        "transport": "http",
        "type": "Synchronous/HTTP"
    }
],
"metadata": {
    "pact-jvm": {
        "version": "4.6.15"
    },
    "pactSpecification": {
        "version": "4.0"
    },
    "plugins": [

    ]
},
"provider": {
    "name": "MicroCredentialsService"
}
}

```



## MicrocredentialRestControllerPactProviderTest:

```
package edu.uoc.epcsd.microcredential.application.rest;

import au.com.dius.pact.provider.junit5.PactVerificationContext;
import
au.com.dius.pact.provider.junit5.PactVerificationInvocationConte
xtProvider;
import au.com.dius.pact.provider.junit5.HttpTestTarget;
import au.com.dius.pact.provider.junitsupport.Provider;
import au.com.dius.pact.provider.junitsupport.State;
import au.com.dius.pact.provider.junitsupport.loader.PactFolder;
import
edu.uoc.epcsd.microcredential.domain.service.MicrocredentialServ
ice;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.TestTemplate;
import org.junit.jupiter.api.extension.ExtendWith;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.boot.web.server.LocalServerPort;

import static org.mockito.Mockito.when;

@SpringBootTest(webEnvironment =
SpringBootTest.WebEnvironment.RANDOM_PORT)
@Provider("MicroCredentialsService")
@PactFolder("src/test/resources/pacts")
@ExtendWith(PactVerificationInvocationContextProvider.class)
class MicrocredentialRestControllerPactProviderTest {

    @LocalServerPort
    private int port;

    @MockBean
    private MicrocredentialService microcredentialService;

    @BeforeEach
    void before(PactVerificationContext context) {
        context.setTarget(new HttpTestTarget("localhost", port,
"/"));
    }

    @TestTemplate
    void pactVerificationTest(PactVerificationContext context) {
```

```

        context.verifyInteraction();
    }

    @State("course 1 exists and is closed")
    void courseExistsAndIsClosed() {

when(microcredentialService.requestCourseMicrocredentials(1L)).t
henReturn(true);
    }

    @State("course 999 does not exist")
    void courseDoesNotExist() {

when(microcredentialService.requestCourseMicrocredentials(999L))
    .thenThrow(new IllegalArgumentException("The
999th course does not exist!"));
    }
}

```

```

Verifying a pact between CourseService and MicroCredentialsService
[Using File src/test/resources/pacts/CourseService-MicroCredentialsService.json]
Given course 1 exists and is closed
successful microcredential creation for course 1
2026-06-10 00:34:41.790 WARN 6320 --- [Thread-2] au.com.dius.pact.core.support.Metrics :
Please note: we are tracking events anonymously to gather important usage statistics like JVM version
and operating system. To disable tracking, set the 'pact_do_not_track' system property or environment
variable to 'true'.

2026-06-10 00:34:42.101 INFO 6320 --- [o-auto-1-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-06-10 00:34:42.102 INFO 6320 --- [o-auto-1-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2026-06-10 00:34:42.104 INFO 6320 --- [o-auto-1-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 2 ms
returns a response which
has status code 201 (OK)
has a matching body (OK)
2026-06-10 00:34:42.331 WARN 6320 --- [main] a.c.d.p.p.DefaultTestResultAccumulator : Not all of the 2 were verified. The following were missing:
2026-06-10 00:34:42.332 WARN 6320 --- [main] a.c.d.p.p.DefaultTestResultAccumulator : failed microcredential creation for missing course 999
2026-06-10 00:34:42.365 INFO 6320 --- [main] rificationStateChangeExtension$Companion : Invoking state change method 'course 999 does not exist':SETUP

Verifying a pact between CourseService and MicroCredentialsService
[Using File src/test/resources/pacts/CourseService-MicroCredentialsService.json]
Given course 999 does not exist
failed microcredential creation for missing course 999
returns a response which
has status code 400 (OK)
has a matching body (OK)
2026-06-10 00:34:42.503 WARN 6320 --- [main] a.c.d.p.p.DefaultTestResultAccumulator : Skipping publishing of verification results as it has been disabled (pact.verifier.publi
shResults is not 'true')
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 7.723 s - in edu.uoc.epcsd.microcredential.application.rest.MicrocredentialRESTControllerPactProviderTest
2026-06-10 00:34:42.567 INFO 6320 --- [ionShutdownHook] j.LocalContainerEntityManagerFactoryBean : Closing JPA EntityManagerFactory for persistence unit 'default'
2026-06-10 00:34:42.567 INFO 6320 --- [ionShutdownHook] .SchemaDropperImpl$DelayedDropActionImpl : HHH000477: Starting delayed evictData of schema as part of SessionFactory shut-down'
Hibernate: drop table if exists microcredential cascade
2026-06-10 00:34:42.577 INFO 6320 --- [ionShutdownHook] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Shutdown initiated...
2026-06-10 00:34:42.581 INFO 6320 --- [ionShutdownHook] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Shutdown completed.
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 10.668 s
[INFO] Finished at: 2026-06-10T00:34:43+02:00
[INFO]

```

Tests pasando correctamente

pom.xml (tanto en Course como en MicroCredential):

```

<dependency>
    <groupId>au.com.dius.pact.provider</groupId>

```

```
        <artifactId>junit5</artifactId>
        <version>4.6.15</version>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter-params</artifactId>
        <scope>test</scope>
    </dependency>
```

## Ejercicio 4: Pruebas de mutación (25 %)

Se ha implementado una nueva clase `CourseValidator` que contiene lógica de negocio para validar diferentes aspectos en la creación de los cursos en el sistema. Actualmente, incluye las siguientes validaciones o criterios de aceptación:

- **Validación de precio:** Rango permitido entre 0 y 10.000€
- **Validación de duración:** Entre 1 y 500 horas
- **Cálculo de descuentos:** Basado en rangos de precio

El código aportado está en el enunciado original del ejercicio.

En este ejercicio debéis realizar los siguientes pasos:

1. Revisa los tests (`testIsValidPrice`, `testIsValidDuration`, `testCalculateDiscount`). ¿Qué porcentaje de cobertura de líneas tiene la clase `CourseValidator`? Describe el significado de éste valor para esta clase.
2. Configura y ejecuta piTest (<https://pitest.org/>) en el proyecto. Explica cómo se configura piTest, qué archivos y fragmentos se modifican, y cómo generar el informe HTML de resultados
3. Revisa el informe HTML de piTest y justifica las métricas siguientes asociadas a las clases `CourseValidatorTest` y `CourseValidator`:
  - a. Mutation coverage
  - b. Line Coverage
  - c. Desglose por mutante. Análisis de cada tipo de mutación
  - d. Mutaciones supervivientes. Cuántos mutantes han sido generados y cuántos de ellos han sobrevivido. Justificar por qué han sobrevivido.
4. Justifica qué pruebas son necesarias para “matar” a los mutantes que han sobrevivido e implementarlas.
5. Re-ejecuta piTest y aporta el nuevo resultado HTML en el que se demuestre que se han eliminado todos los mutantes supervivientes.

### Notas:

El código mostrado en el enunciado no está en la solución de la PRAC2, debéis copiar su contenido en vuestro repositorio, dentro del directorio `src/test`.

- 
1. Si trazamos la ejecución de los tests proporcionados, podemos comprobar que todas las líneas ejecutables (todas las evaluaciones lógicas y todas las sentencias `return`) se visitan al menos una vez; es decir, el porcentaje de cobertura de líneas de la clase `CourseValidator` con las pruebas actuales es del 100%.:
    - **`isValidPrice`:** Se prueba con 5000L (ejecuta el `return` final) y con null (ejecuta el `return false` dentro del `if`).

- **isValidDuration:** Se prueba con 100L (ejecuta el return final) y con null (ejecuta el return false dentro del if).
- **calculateDiscount:** Se prueba con 7000L (entra en el if >= 5000), con 3000L (entra en el else if >= 2000), con 1000L (entra en el else) y con null (entra en el primer if por ser nulo y devuelve 0).

Tener un 100% de cobertura de líneas significa que el flujo de ejecución de los tests ha "pisado" cada instrucción del código fuente al menos una vez. Sin embargo, en el contexto de esta clase y de la ingeniería de software en general, este valor trae consigo importantes matices que debes considerar: Una de ellas es la falsa sensación de seguridad, ya que aunque todas las líneas se ejecutan, no todas las condiciones se han probado. Por ejemplo, en la línea `if (price == null || price < 0)`, el test pasa un valor null, lo que hace que la primera parte de la condición sea verdadera y se ejecute el `return 0`; Sin embargo, no existe ningún test que compruebe qué pasa si el precio es negativo (ej. -100L). La línea está cubierta, pero la condición de precio negativo jamás ha sido evaluada.

2. El único archivo que necesitamos modificar es el `pom.xml` de Courses. Tenemos que añadir el plugin de PiTest dentro de la sección `<properties>` y también como `<plugin>`:

```
<properties>
  <java.version>11</java.version>
  <allure.version>2.35.2</allure.version>
  <pitest.version>1.15.8</pitest.version>

  <pitest.junit5.plugin.version>1.2.1</pitest.junit5.plugin.ver
  sion>
</properties>
...
    <plugin>
      <groupId>org.pitest</groupId>
      <artifactId>pitest-maven</artifactId>
      <version>${pitest.version}</version>
      <configuration>
        <targetClasses>

<param>edu.uoc.epcsd.course.domain.validator.*</param>
        </targetClasses>
        <targetTests>

<param>edu.uoc.epcsd.course.domain.validator.*</param>
        </targetTests>
        <outputFormats>
          <param>HTML</param>
        </outputFormats>
      </configuration>
    </plugin>
  </dependencies>
</project>
```

```

<reportsDirectory>${project.build.directory}/pit-reports</reportsDirectory>

    </configuration>
    <dependencies>
        <dependency>
            <groupId>org.pitest</groupId>

<artifactId>pitest-junit5-plugin</artifactId>

<version>${pitest.junit5.plugin.version}</version>
        </dependency>
    </dependencies>
</plugin>

```

Una vez añadido al `pom.xml`, podemos ejecutarlo con el comando `mvn test pitest:mutationCoverage`:

```

=====
- Timings
=====
> pre-scan for mutations : < 1 second
> scan classpath : < 1 second
> coverage and dependency analysis : < 1 second
> build mutation tests : < 1 second
> run mutation analysis : 1 seconds
-----
> Total : 2 seconds
-----
- Statistics
=====
>> Line Coverage (for mutated classes only): 14/14 (100%)
>> Generated 24 mutations Killed 24 (100%)
>> Mutations with no coverage 0. Test strength 100%
>> Ran 24 tests (1 tests per mutation)
Enhanced functionality available at https://www.arcmutate.com/

!! The following issues were detected during the run !!
Project uses Spring, but the Arcmutate Spring plugin is not present (https://docs.arcmutate.com/docs/spring.html)
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 16.370 s
[INFO] Finished at: 2026-06-10T00:59:37+02:00
[INFO] -----

```

mutationCoverage ejecutado correctamente con cobertura del 100%

Al ejecutar este comando, también se genera el informe HTML con los resultados, el cual se encuentra en el directorio `target/pit-reports/<fecha-y-hora>/index.html`:

# Pit Test Coverage Report

## Project Summary

| Number of Classes | Line Coverage | Mutation Coverage | Test Strength |
|-------------------|---------------|-------------------|---------------|
| 1                 | 100% 14/14    | 100% 24/24        | 100% 24/24    |

## Breakdown by Package

| Name                                                  | Number of Classes | Line Coverage | Mutation Coverage | Test Strength |
|-------------------------------------------------------|-------------------|---------------|-------------------|---------------|
| <a href="#">edu.uoc.epcsd.course.domain.validator</a> | 1                 | 100% 14/14    | 100% 24/24        | 100% 24/24    |

Report generated by [PIT](#) 1.15.8

Enhanced functionality available at [arcmutate.com](#)

Resultados de `mvn test pitest:mutationCoverage`

- Basándonos en el informe de PiTest y en el código de prueba actualizado que has proporcionado, aquí tienes la justificación detallada de las métricas solicitadas.

### Mutation Coverage - Métrica: 100% (24/24 mutantes asesinados)

PiTest inyectó 24 mutantes en el código para comprobar si los tests fallarían. Que la métrica sea del 100% significa que la suite de pruebas `CourseValidatorTest` es lo suficientemente estricta y exhaustiva como para hacer fallar a los 24 mutantes. No hay "falsos positivos" en la cobertura.

### Line Coverage - Métrica: 100% (14/14 líneas)

Esta métrica indica que la ejecución de los tests pasa por cada una de las 14 líneas de código ejecutable de la clase `CourseValidator`. Al haber introducido pruebas para casos válidos, inválidos y valores nulos, nos aseguramos de que todos los caminos de ejecución posibles de los métodos `isValidPrice`, `isValidDuration` y `calculateDiscount` se visitan.

### Desglose por mutante

Para alcanzar el 24/24, PiTest ha aplicado diferentes operadores de mutación en la lógica de `CourseValidator`, y los tests han reaccionado a cada uno de ellos:

- Conditionals Boundary Mutator:** Cambia los operadores relacionales. Por ejemplo, transforma `price >= 5000` en `price > 5000`. En este caso, el mutante falla porque hemos incluido pruebas exactas en las fronteras. El test `validator.calculateDiscount(5000L)` espera un 20. Si el mutante cambia el código a `>`, 5000 pasaría a la rama del `else if (price >= 2000)` y devolvería 10. El test falla y el mutante muere. Lo mismo ocurre al probar los valores exactos 0L, 1L, 500L, y 10000L.
- Negate Conditionals Mutator:** Invierte la lógica booleana. Cambia `==` por `!=`, o `>=` por `<`. Por ejemplo, transforma `if (price == null)` en `if (price != null)`. El mutante falla porque tenemos aserciones explícitas para valores nulos y valores fuera de rango (como -1L o

10001L). Si el condicional se invierte, el método devuelve un valor incorrecto que las aserciones capturan inmediatamente.

- **Return Values Mutator:** Altera directamente el valor devuelto. Transforma `return true;` en `return false;`, o `return 20;` en `return 0;`. El mutante falla porque estamos usando `isTrue()`, `isFalse()` y comprobaciones numéricas estrictas (`isEqualTo(20)`). Un mutante que cambie el resultado final nunca pasará estas aserciones.

### Mutaciones supervivientes

Mutantes generados: 24

Mutantes supervivientes: 0

No ha sobrevivido ninguno. Estamos teniendo en cuenta los valores límite correctamente en todos los escenarios posibles.